LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 05 – Relasi Antar Kelas



Nama: CALYA ALESITA PUTRI IRAWAN

NIM: 103132400020

Kelas: PS1DS-05-REG-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Pemrograman Berorientasi Objek (PBO) merupakan konsep pemrograman yang menggunakan class dan object untuk menyusun suatu program. Dalam PBO, beberapa class dapat saling berhubungan untuk menjalankan fungsi tertentu. Relasi antar kelas terdiri dari komposisi, agregasi, dan asosiasi.

Komposisi merupakan hubungan antarobjek yang kuat, ketika suatu objek memiliki dan membentuk objek lain sebagai bagian dari dirinya. Agregasi merupakan hubungan ketika suatu objek menampung atau menggunakan objek lain yang dapat dibuat dan tetap ada secara terpisah. Sementara itu, asosiasi merupakan hubungan antara dua objek yang saling berinteraksi untuk menjalankan suatu fungsi. Pemahaman mengenai ketiga relasi tersebut membantu dalam merancang program Java yang terstruktur dan sesuai dengan konsep Pemrograman Berorientasi Objek.

B. Guided

1. Guided 1

```java
package modul5.guided.guided1;

public class Mahasiswa {
    private String nim;
    private String nama;
    private int nilai;

    public void setNama(String nama){
        this.nama = nama;
    }
    public void setNim(String nim){
        this.nim = nim;
    }
    public void setNilai(int nilai){
        this.nilai = nilai;
    }
    public String getNama(){
        return this.nama;
    }
    public int getNilai(){
        return this.nilai;
    }
    public String getNim(){
        return this.nim;
    }
}
```

```java
package modul5.guided.guided1;
```

```java
public class Dosen {
    private String kodeDosen;
    private String namaDosen;

    //Setter
    public void setKodeDosen(String kodeDosen){
        this.kodeDosen = kodeDosen;
    }
    public void setNamaDosen(String namaDosen){
        this.namaDosen = namaDosen;
    }
    //Getter
    public String getKodeDosen(){
        return this.kodeDosen;
    }
    public String getNamaDosen(){
        return this.namaDosen;
    }

    public void giveScore(Mahasiswa s, int nilai){
        s.setNilai(nilai);
    }
    public int getScore(Mahasiswa s){
        return s.getNilai();
    }

    public static void main(String[] args) {
        Mahasiswa m = new Mahasiswa();

        m.setNim("130118383");
        m.setNama("Budi");

        Dosen d = new Dosen();
        d.giveScore(m, 90);

        System.out.println("Nim :"+m.getNim());
        System.out.println("Nama :"+m.getNama());
        System.out.println("Nilai :"+d.getScore(m));
    }
}
```

Output :

```
Nim :130118383
Nama :Budi
Nilai :90
```

2. Agregasi

```java
package modul5.guided.agregasi;

public class Mahasiswa {
    private String NIM, Nama;

    public Mahasiswa(String NIM, String Nama) {
        this.NIM = NIM;
        this.Nama = Nama;
    }

    public String getNIM() {
        return NIM;
    }

    public String getNama() {
        return Nama;
    }

    public void setNIM(String NIM) {
        this.NIM = NIM;
    }

    public void setNama(String Nama) {
        this.Nama = Nama;
    }
}
```

```java
package modul5.guided.agregasi;

public class Jurusan {
    private static int jumlahMahasiswa;
    private String kodeJurusan, namaJurusan;
    private Mahasiswa[] daftarMahasiswa = new Mahasiswa[32];

    // Constructor
    public Jurusan(String kodeJurusan, String namaJurusan) {
        this.kodeJurusan = kodeJurusan;
        this.namaJurusan = namaJurusan;
    }
```

```
    // Behavior
    public void addMahasiswa(Mahasiswa m) {
        this.daftarMahasiswa[jumlahMahasiswa] = m;
        jumlahMahasiswa++;
    }

    public void displayMahasiswa() {
        System.out.println("Kode Jurusan: " + this.kodeJurusan);
        System.out.println("Nama Jurusan: " + this.namaJurusan);

        System.out.println("Daftar Mahasiswa:");

        for (int i = 0; i < jumlahMahasiswa; i++) {
            System.out.println((i + 1) + ". "
                    + daftarMahasiswa[i].getNIM() + " "
                    + daftarMahasiswa[i].getNama());
        }
    }

    // Getter
    public String getKodeJurusan() {
        return kodeJurusan;
    }

    public String getNamaJurusan() {
        return namaJurusan;
    }

    // Setter
    public void setKodeJurusan(String kodeJurusan) {
        this.kodeJurusan = kodeJurusan;
    }

    public void setNamaJurusan(String namaJurusan) {
        this.namaJurusan = namaJurusan;
    }
}
```

```
package modul5.guided.agregasi;

public class Main {
    public static void main(String[] args) {
        runGuided2();
    }
```

```
    public static void runGuided2() {
        Mahasiswa m1 = new Mahasiswa("103132400020", "Calya");
        Mahasiswa m2 = new Mahasiswa("103132400021", "John Doe");

        Jurusan prodi = new Jurusan("SD", "Sains Data");

        prodi.addMahasiswa(m1);
        prodi.addMahasiswa(m2);

        prodi.displayMahasiswa();
    }
}
```

Output :

```
Kode Jurusan: SD
Nama Jurusan: Sains Data
Daftar Mahasiswa:
1. 103132400020 Calya
2. 103132400021 John Doe
```

3. Komposisi

```
package modul5.guided.komposisi;

public class Komputer {
    private Processor processor;
    private Monitor monitor;
    private Mouse mouse;

    public Komputer(String brandCpu, int kecepatanCpu, String
brandMonitor, String brandMouse, String tipeMouse){
        this.processor = new Processor(brandCpu, kecepatanCpu);
        this.monitor = new Monitor(brandMonitor);
        this.mouse = new Mouse(brandMouse, tipeMouse);
    }

    public void displaySpecKomputer() {
        System.out.println("=== SPESIFIKASI KOMPUTER ===");
        this.processor.displaySpec();
        this.monitor.DisplaySpec();
        this.mouse.displaySpec();
    }

}
```

```java
package modul5.guided.komposisi;

public class Monitor {
    private String brand;

    public Monitor(String brand){
        this.brand = brand;
    }

    public void DisplaySpec() {
        System.out.println(this.brand);
    }
}
```

```java
package modul5.guided.komposisi;

public class Mouse {
    private String brand, tipe;

    public Mouse(String brand, String tipe){
        this.brand = brand;
        this.tipe = tipe;
    }

    public void displaySpec(){
        System.out.println(this.brand + ", " + this.tipe);
    }
}
```

```java
package modul5.guided.komposisi;

public class Processor {
     private String brand;
    private int kecepatan;

    //constructor
    public Processor(String brand,int kecepatan){
        this.brand = brand;
        this.kecepatan = kecepatan;
    }

    //behaviour
    public void displaySpec(){
        System.out.println(this.brand + ", " + this.kecepatan);
    }
}
```

Output :

```
=== SPESIFIKASI KOMPUTER ===
Intel Core i5, 3200
Samsung
Logitech, Wireless
```

C. Unguided

1. Soal Unguided 1

Buatlah program Java untuk sistem manajemen pesanan cafe dengan menerapkan komposisi, agregasi, dan asosiasi serta konsep enkapsulasi, constructor, dan array berkapasitas tetap.

1. Komposisi: Buat class RincianHarga untuk menghitung harga dasar dan pajak 10%, serta class ItemPesanan yang memiliki objek RincianHarga di dalam constructor-nya.

2. Agregasi: Buat class Pesanan untuk menyimpan nomor meja dan daftar item pesanan menggunakan array, serta menyediakan method untuk menambahkan dan menampilkan item.

3. Asosiasi: Buat class Kasir dengan atribut nama kasir dan method prosesPembayaran() untuk menghitung subtotal, total pajak, dan total pembayaran dari objek Pesanan.

4. Pengujian: Buat class Main untuk membuat objek kasir bernama Budi, tiga item pesanan (Iced Americano Rp18.000, Beef Croissant Rp25.000, dan Matcha Latte Rp22.000), serta pesanan Meja 4. Tambahkan ketiga item ke pesanan, tampilkan daftar pesanan, lalu proses pembayaran dan tampilkan struk pembayaran.

Jawaban :

```java
package modul5.unguided;

public class RincianHarga {
    private double hargaDasar;
    private double pajak;

    public RincianHarga(double hargaDasar) {
        this.hargaDasar = hargaDasar;
        this.pajak = hargaDasar * 0.10;
    }

    public double getHargaDasar() {
        return hargaDasar;
    }
```

```
    public double getPajak() {
        return pajak;
    }

    public double getHargaTotal() {
        return hargaDasar + pajak;
    }
}
```

```
package modul5.unguided;

public class ItemPesanan {
    private String namaItem;
    private RincianHarga rincian;

    public ItemPesanan(String namaItem, double hargaDasar) {
        this.namaItem = namaItem;
        this.rincian = new RincianHarga(hargaDasar);
    }

    public String getNamaItem() {
        return namaItem;
    }

    public RincianHarga getRincian() {
        return rincian;
    }
}
```

```
package modul5.unguided;

public class Pesanan {
     private int nomorMeja;
    private ItemPesanan[] daftarItem;
    private int jumlahItem;

    public Pesanan(int nomorMeja) {
        this.nomorMeja = nomorMeja;
        this.daftarItem = new ItemPesanan[5];
        this.jumlahItem = 0;
    }

    public void tambahItem(ItemPesanan item) {
        if (jumlahItem < daftarItem.length) {
            daftarItem[jumlahItem] = item;
            jumlahItem++;
        } else {
```

```
            System.out.println("Pesanan sudah penuh.");
        }
    }

    public void tampilkanPesanan() {
        System.out.println("=== DAFTAR PESANAN CAFE ===");
        System.out.println("Nomor Meja: " + nomorMeja);

        for (int i = 0; i < jumlahItem; i++) {
            ItemPesanan item = daftarItem[i];
            RincianHarga rincian = item.getRincian();

            System.out.println((i + 1) + ". " + item.getNamaItem()
                    + " - Rp " + rincian.getHargaTotal()
                    + " (Harga: " + rincian.getHargaDasar()
                    + ", Pajak: " + rincian.getPajak() + ")");
        }

        System.out.println();
    }

    public int getNomorMeja() {
        return nomorMeja;
    }

    public ItemPesanan[] getDaftarItem() {
        return daftarItem;
    }

    public int getJumlahItem() {
        return jumlahItem;
    }
}
```

```
package modul5.unguided;

public class Kasir {
    private String namaKasir;

    public Kasir(String namaKasir) {
        this.namaKasir = namaKasir;
    }

    public void prosesPembayaran(Pesanan pesanan) {
        double subtotal = 0;
        double totalPajak = 0;
```

```
        ItemPesanan[] daftarItem = pesanan.getDaftarItem();

        for (int i = 0; i < pesanan.getJumlahItem(); i++) {
            RincianHarga rincian = daftarItem[i].getRincian();
            subtotal += rincian.getHargaDasar();
            totalPajak += rincian.getPajak();
        }

        System.out.println("=== STRUK PEMBAYARAN KASIR ===");
        System.out.println("Kasir Bertugas : " + namaKasir);
        System.out.println("Pesanan        : Meja " +
pesanan.getNomorMeja());
        System.out.println("Jumlah Item    : " +
pesanan.getJumlahItem());
        System.out.println("Total Subtotal : Rp " + subtotal);
        System.out.println("Total Pajak    : Rp " + totalPajak);
        System.out.println("TOTAL BAYAR    : Rp " + (subtotal +
totalPajak));
        System.out.println("==============================");
    }
}
```

```
package modul5.unguided;

public class Main {
    public static void main(String[] args) {
        Kasir kasir = new Kasir("Budi");

        ItemPesanan item1 =
                new ItemPesanan("Iced Americano", 18000);
        ItemPesanan item2 =
                new ItemPesanan("Beef Croissant", 25000);
        ItemPesanan item3 =
                new ItemPesanan("Matcha Latte", 22000);

        Pesanan pesanan = new Pesanan(4);

        pesanan.tambahItem(item1);
        pesanan.tambahItem(item2);
        pesanan.tambahItem(item3);

        pesanan.tampilkanPesanan();
        kasir.prosesPembayaran(pesanan);
    }
}
```

Output :

```
=== DAFTAR PESANAN CAFE ===
Nomor Meja: 4
1. Iced Americano - Rp 19800.0 (Harga: 18000.0, Pajak: 1800.0)
2. Beef Croissant - Rp 27500.0 (Harga: 25000.0, Pajak: 2500.0)
3. Matcha Latte - Rp 24200.0 (Harga: 22000.0, Pajak: 2200.0)

=== STRUK PEMBAYARAN KASIR ===
Kasir Bertugas : Budi
Pesanan        : Meja 4
Jumlah Item    : 3
Total Subtotal : Rp 65000.0
Total Pajak    : Rp 6500.0
TOTAL BAYAR    : Rp 71500.0
============================
```

Penjelasan :

Pogram system manajemen pesanan café ini menerapkan relasi komposisi, agregasi, dan asosiasi. Relasi komposisi diterapkan pada *class* ItemPesanan dan RincianaHarga, yaitu objek rincian harga dibuat di dalam *constructor* ItemPesanan. Relasi agregasi diterapkan pada *class* Pesanan yang menyimpan beberapa objek ItemPesanan menggunakan *array*. Sementara itu, relasi asosiasi diterapkan pada *class* Kasir yang memproses pembayaran dengan menerima objek Pesanan sebagai parameter method prosesPembayaran().

Program dijalankan melalui class Main dengan membuat objek kasir bernama Budi, tiga item pesanan, dan pesanan untuk meja nomor 4. Kemudian menampilkan daftar pesanan beserta harga dasar, pajak sebesar 10%, dan harga total setiap item, serta menghasilkan struk pembayaran. Berdasarkan hasil pengujian, diperoleh subtotal sebesar Rp65.000, total pajak Rp6.500, dan total pembayaran Rp71.500. Hasil tersebut menunjukkan bahwa program berhasil menjalankan fungsi yang telah dirancang sesuai dengan ketentuan soal.

D. Kesimpulan

Berdasarikan Praktikum Modul 5 tentang Relasi Antar Kelas, dapat disimpulkan bahwa relasi komposisi. Agregasi, dan Asosiasi memiliki cara penerapan yang berbeda dalam pemrograman beriorientasi objek menggunakan Java. Pada Guided, konsep relasi antar kelas dipelajari melalui contoh program, seperti hubungan antara komputer dengan komponennya serta hubungan antara jurusan dan mahasiswa. Pada Unguided, konsep tersebut diterapkan dalam program manajemen pesanan café melalui *class* RincianHarga, ItemPesanan, Pesanan, dan Kasir. Dalam praktikum ini juga membantu memahami *class, object, constructor*, enkapsulasi, *method*, dan *array*. Dan memberikan pemahaman tentang cara menghubungkan beberapa *class* agar dapat bekerja sama dalam membangun sebuah program.